You Asked

Let's now say I go with the ML option and use the Discover Hack option and Users adding tags to movies, shows, books, music, etc. as training data for the ML

Gemini 3.1 Pro

This is a massive leap forward in systems design! By combining official API metadata (the "Discover Hack") with user-generated custom tags, you are effectively building what the industry calls a **Taste Graph**. This is exactly how Spotify and Netflix bridge the gap between "what the creator says this is" and "how the audience actually feels about it."

Since you are hosting the frontend on GitHub Pages but using Supabase as your database, here is the reality of how you can architect this ML pipeline without needing a heavy, expensive Python backend.

The Strategy: Vector Similarity via Supabase (`pgvector`)

As a CS student, you have likely heard of **Embeddings** and **Vector Databases**. This is the modern, serverless way to handle Machine Learning recommendations. Supabase natively supports an extension called `pgvector`, which allows you to do complex ML similarity math directly inside your PostgreSQL database.

Here is how your pipeline would work:

Phase 1: The Hybrid Dataset (Data Collection)

Your Supabase database will become the central brain. When a user looks at a movie (e.g., *Everything Everywhere All At Once*):

1. **The API Layer:** You fetch the official TMDB tags (Action, Sci-Fi, Comedy).
2. **The User Layer:** Users add their own tags to it in your app (e.g., "Mind-Bending," "Family Trauma," "Visually Stunning").
3. **The Aggregation:** You store an aggregated profile of this movie in Supabase that combines *both*. The more a user tag is applied by different users, the higher its "weight" becomes.

Phase 2: Generating the "Vibe Vector" (The ML Part)

Machine Learning models don't read English; they read arrays of numbers called vectors. You can use a free, lightweight embedding model (like OpenAI's `text-embedding-3-small` or an open-source

HuggingFace model via an API) to turn that massive list of tags into a mathematical vector.

- *EEAAO Tags:* `["Action", "Sci-Fi", "Comedy", "Mind-Bending", "Family Trauma"]`
- *EEAAO Vector:* `[0.12, -0.45, 0.88, 0.02, ...]` (Imagine a 384-dimensional array).

You store this vector in a special Supabase column.

Phase 3: The "Discover" Query (Execution)

Now, a user comes to your app and selects 3 movies they like.

1. **Frontend:** Your GitHub Pages JS fetches the vectors for those 3 movies from Supabase.
2. **Math:** You average those 3 vectors together in JavaScript to create a "Target Vibe Vector."
3. **The Magic Query:** You send this single target vector to Supabase. Using `pgvector`, Supabase calculates the **Cosine Similarity** (how mathematically close the vectors are in 384-dimensional space) and returns the top 5 closest matches.

Why this is the perfect architecture for you:

- **No Backend Required:** You don't need to spin up a Flask or Django server to run ML models. Supabase handles the heavy lifting via SQL queries.
- **Crowdsourced Intelligence:** Because the vectors are generated from *user tags* as well as official tags, your app will start to learn that "Blade Runner" and "Cyberpunk 2077" share a vibe, even if they cross media types (Movies vs. Video Games).
- **Cross-Media Recommendations:** This solves your Book/Music problem perfectly. If a user tags a Last.fm album as "Dystopian, Gritty" and an OpenLibrary book as "Dystopian, Gritty," their vectors will be incredibly similar. **You can now recommend a Book based on a Music Album.**

How to pitch this in an interview / portfolio:

> *"Catalogd uses a hybrid recommendation engine. It solves the cold-start problem by utilizing standard API metadata, but continuously improves its accuracy by utilizing user-generated tags to build a cross-media taste graph, stored as vector embeddings in Supabase for high-performance cosine similarity search."*

It is a fantastic idea. If you want to go down this route, your next step would be enabling the `pgvector` extension in your Supabase dashboard and figuring out which embedding API you want to use to turn your tags into numbers. How comfortable are you with SQL?